

# Esame OS Internals (Cabodi)

Data: 17/01/2022

---

*Nota Generale: TUTTE LE RISPOSTE SI/NO VANNO MOTIVATE. PER LE RISPOSTE NUMERICHE, SONO RICHIESTI SIA I RISULTATI CHE I PASSAGGI INTERMEDI (O FORMULE) RILEVANTI.*

## 1 SECTION 1: THEORETICAL (Silberschatz)

### Domanda 1 (Paginazione, Working Set e Codice C)

**Contesto:** Si consideri il seguente frammento di codice:

```
#define N 2048
float M[N][2*N], V[2*N*N], x;
int i, j, t;

for (t=0; t < 2*N*N; t++) {
    i = t / (2*N);
    j = t % (2*N);
    x = M[i][j];
    V[t] = x;
    M[i][j] = x * 2;
}
```

Il codice macchina generato da tali istruzioni viene eseguito su un sistema con gestione della memoria basata su demand paging (paginazione a richiesta), pagine da 2 KB, utilizzando una politica di sostituzione pagine working set (versione esatta) con  $\delta = 10$ . Si sa che:

- la dimensione di un `float` è di 32 bit;
- il segmento di codice (istruzioni in codice macchina) ha dimensioni inferiori a una pagina;
- `M` e `V` sono allocati ad indirizzi logici contigui (prima `M`, poi `V`), a partire dall'indirizzo logico `0xDB28A800`;
- la matrice `M` è allocata seguendo la strategia "row major", ovvero per righe (prima riga, seguita da seconda riga, ecc..).

**Si risponda alle seguenti domande:**

- A) Quante pagine (e frame) sono necessarie per contenere la matrice e il vettore?
- A) Supponiamo che le variabili `i`, `j`, `t` e `x` siano allocate in registri della CPU (accedendo a tali registri non si fa quindi alcun accesso in memoria RAM), quanti accessi totali a memoria  $N_T = N_W + N_R$  ( $N_R$  per la lettura e  $N_W$  per la scrittura di dati) produce il programma proposto (non vanno conteggiati gli accessi a istruzioni)?

A) Sia  $N_T$  la quantità totale di riferimenti a dati in memoria (omettiamo per semplicità il fetch delle istruzioni) e sia  $N_L$  il numero di riferimenti (tra gli  $N_T$  totali) a una pagina già letta nei precedenti 10 accessi. Definiamo come località del programma per i dati il rapporto  $L = N_L/N_T$ . Si calcoli la località del programma proposto.

A) Si calcoli il numero di page fault generati dal programma proposto. (Motivare la risposta).

---

## Domanda 2 (Disk Scheduling e SSTF)

**Contesto:** Si consideri un "disk scheduler" che gestisce richieste di accesso a blocchi logici su un disco di tipo HDD. La funzione `enqNextBlk` (abbreviata `eNB`) viene utilizzata per mettere nella coda la richiesta di accesso al blocco `blk`. La funzione `deqNextBlk` (`dNB`) viene chiamata per ottenere il nuovo blocco a cui fare accesso (il blocco schedulato) ogni volta che il disco è nuovamente pronto per una operazione. Per semplicità, non si distingue tra operazioni di lettura e scrittura.

Supponendo la seguente sequenza di chiamate, con coda di schedulazione inizialmente vuota e blocco corrente head (posizione iniziale della testina di lettura/scrittura) inizializzato al valore 500:

```
eNB (3050)
eNB (5150)
N = dNB()
eNB (1000)
eNB (1050)
N = dNB()
eNB (1200)
N = dNB()
N = dNB()
N = dNB()
```

**Si risponda alle seguenti domande:**

A) Si indichino i valori successivi di head e di N (accessi a blocchi su disco effettuati) e si rappresenti la coda dopo ogni chiamata a `enqNextBlk` (`eNB`). Si adotti una politica di schedulazione SSTF (Shortest Seek Time First), nella quale `dNB` ritorna il blocco più vicino (in una qualunque delle due direzioni) alla testina di lettura scrittura (head).

A) Si calcoli la lunghezza del percorso totale della testina di lettura/scrittura, misurato in numero di cilindri (ogni cilindro contiene 50 blocchi).

---

## Domanda 3 (Link dinamico, IPT e TLB)

**Contesto:** Si risponda alle seguenti domande sulla gestione della memoria:

A) Si consideri il caricamento dinamico (dynamic loading) e il link dinamico (dynamic linking). È possibile caricare dinamicamente un programma senza che sia necessario il dynamic linking? Il dynamic linking richiede che un programma sia anche caricabile dinamicamente (dynamic loading)?

- A)** Si spieghi brevemente perché un'Inverted Page Table necessiti di una tabella di HASH. Perché la soluzione di IPT + tabella di HASH è diversa da una soluzione con PT basata solamente su tabella di Hash?
- A)** Si consideri una CPU dotata di TLB: la TLB può contenere entry di più processi simultaneamente o è vincolata a contenere entry per un solo processo?

## 2 SECTION 2: OS161

### Domanda 4 (System Call `waitpid`)

**Contesto:** Sia dato un sistema OS161. Si vuol realizzare il supporto (parziale, in quanto non si gestisce il parametro `flags`) per la system call `waitpid`, di cui si fornisce il prototipo:

```
pid_t waitpid (pid_t pid, int *returncode, int flags);
```

Si è già realizzata una funzione `proc_wait`, avente prototipo:

```
int proc_wait (struct proc *proc);
```

che dato il puntatore al descrittore di un processo, ne attende la fine (del processo) e ne ritorna lo stato di terminazione.

**Quesito:** Si realizzi una funzione `sys_waitpid`, che possa essere richiamata nella syscall mediante:

```
case SYS_waitpid:
    retval = sys_waitpid(tf->tf_a0, (userptr_t)tf->tf_a1);
    break;
```

*ATTENZIONE:* Di una eventuale tabella dei processi, è sufficiente fornire la dichiarazione e l'uso all'interno di `sys_waitpid`, nonché una breve descrizione a parole. Non è necessaria la gestione della tabella al boot e alla creazione/distruzione di un processo.

---

### Domanda 5 (Strutture `uio` e `iovec`)

**Contesto:** Si consideri, in un sistema OS161, le definizioni delle `struct uio` e `iovec` qui riportate (`iovec` in forma semplificata ma equivalente all'originale):

```
struct uio {
    struct iovec *uio_iov;
    unsigned uio_iovcnt;
    off_t uio_offset;
    size_t uio_resid;
    enum uio_seg uio_segflg;
    enum uio_rw uio_rw;
    struct addrspace *uio_space;
};

struct iovec {
    void *iov_base;
    size_t iov_len;
};
```

**Si risponda alle seguenti domande:**

- B) Spiegare brevemente la funzione svolta dai campi `uio_segflg`, `uio_rw` e `uio_space`.
- B) Nel realizzare la system call `sys_write`, ci si aspetta che il campo `uio_segflg` indichi un segmento kernel o user? (Motivare).

- B) Si dica poi perchè la `struct iovec`, puntata dal campo `uio_iov`, è una struct esterna. Sarebbe possibile evitare la struct aggiuntiva inserendo direttamente i campi `iov_base` e `iov_len` nella `struct uio`, come riportato qui sotto? (Motivare la risposta).

```
struct uio {  
    void *iov_base;  
    size_t iov_len;  
    unsigned uio_iovcnt;  
    off_t uio_offset;  
    size_t uio_resid;  
    enum uio_seg uio_segflg;  
    enum uio_rw uio_rw;  
    struct addrspace *uio_space;  
};
```